Programowanie równoległe i rozproszone

Tematy projektów 2026/2027

Projekty są przeznaczone do realizacji indywidualnej lub w zespołach maksymalnie dwuosobowych. Katalog obejmuje 41 tematów: 11 ogólnych, 12 dla Cyberbezpieczeństwa, 12 dla Data Science oraz 6 dla specjalności Systemy Inteligentne i Rozszerzona Rzeczywistość. Nie jest koniecznym wybieranie tematu pod swoją specjalność.

Wymagania wspólne

Każdy projekt powinien zawierać implementację referencyjną oraz trzy wersje wykorzystujące OpenMP, MPI i CUDA lub OpenCL. Dopuszczalne są odpowiedniki tych technologii w wybranym języku po uzgodnieniu z prowadzącym. Wszystkie wersje muszą przetwarzać te same dane i realizować ten sam cel obliczeniowy. Różnice algorytmów i dokładności należy jawnie opisać.

Własna implementacja powinna obejmować wskazany w temacie rdzeń obliczeń. Biblioteki mogą służyć do odczytu danych, wizualizacji, pomocniczych operacji oraz jako zoptymalizowane punkty odniesienia. Gdy dopuszczono bibliotekę dla części obliczeń, w raporcie trzeba wskazać jej udział i wyodrębnić wkład własny. Rozbudowany interfejs graficzny nie jest wymagany; wystarczy poprawnie udokumentowany interfejs wiersza poleceń.

Zakres dla jednej i dwóch osób

Każda karta określa zakres podstawowy i konkretny dodatek dla zespołu dwuosobowego. W obu przypadkach obowiązują trzy technologie. Zespół jednoosobowy ogranicza liczbę wariantów i scenariuszy zgodnie z kartą. Zespół dwuosobowy realizuje również wskazane porównanie lub usprawnienie. Podział pracy należy przedstawić w dokumentacji; obie osoby powinny rozumieć całość rozwiązania.

Dokumentacja projektu

Do projektu należy dołączyć kod, instrukcję budowania i uruchomienia, opis danych, testy poprawności, konfigurację komputerów i oprogramowania, surowe wyniki pomiarów oraz skrypty tworzące wykresy. Sprawozdanie powinno wyjaśniać podział obliczeń, synchronizację, komunikację i najważniejsze decyzje implementacyjne. Wnioski muszą wynikać z pomiarów, także wtedy, gdy GPU lub MPI nie daje przyspieszenia.

Rozwinięcie naukowe

Ostatnia część każdej karty proponuje pytanie badawcze do rozwinięcia w artykuł lub pracę magisterską. Nie jest to obowiązkowy zakres zaliczenia. Dalsza praca wymaga przeglądu literatury, mocnych metod odniesienia, weryfikacji nowości i poszerzonej walidacji. Samo wykonanie trzech wersji znanego algorytmu nie przesądza o wkładzie naukowym.

Pomiary i organizacja pracy

Poprawność i porównywalność

Przed pomiarami należy porównać wyniki z referencją sekwencyjną, sprawdzić przypadki brzegowe i ustalić tolerancję dla liczb zmiennoprzecinkowych. W eksperymentach losowych zapisywać ziarna i zapewnić rozłączne strumienie generatorów. W uczeniu maszynowym oddzielić dane treningowe, walidacyjne i testowe; przygotowanie cech i dobór progów nie mogą korzystać z danych testowych.

Czas i skalowanie

Osobno mierzyć czas rdzenia obliczeń oraz pełny czas przetwarzania, obejmujący istotne transfery, komunikację i wejście/wyjście. Wykonać rozgrzewkę oraz co najmniej pięć niezależnych pomiarów konfiguracji. Pokazać medianę i rozrzut, a dla metod losowych także wyniki wielu ziaren. Operacje GPU zsynchronizować przed końcem pomiaru.

Dla CPU i MPI wykonać wykresy przyspieszenia $S(p)=T(1)/T(p)$ i efektywności $E(p)=S(p)/p$ przy stałym problemie i tej samej metodzie. Podać liczbę wątków, procesów, rdzeni i węzłów. Wynik MPI na jednym komputerze należy oznaczyć jako lokalny; nie stanowi on pomiaru skalowalności klastra. Gdy MPI dzieli niezależne zadania, raportować przepustowość wsadu, odróżniając ją od czasu pojedynczego zadania.

Dla GPU pokazać czas, przepustowość oraz wpływ rozmiaru danych i konfiguracji bloków. Liczby wątków GPU nie używać jako odpowiednika liczby rdzeni CPU w formule efektywności. Przyspieszenie względem CPU wymaga jawnego wskazania referencji. Zestawienie najlepszych wyników musi zachować tę samą jakość i zakres mierzonych operacji. P95 oznacza 95. percentyl opóźnienia i wymaga wielu obserwacji zadań, nie tylko pięciu pomiarów całego programu.

Etapy realizacji i ocena

Proponowany podział semestru: tygodnie 1–2 – wybór tematu, zakres i dostęp do sprzętu; 3–4 – referencja oraz testy; 5–7 – OpenMP i MPI; 8–10 – GPU; 11–12 – eksperymenty; 13–14 – raport i obrona. Konkretne terminy konsultacji i oddania ustala prowadzący. Proponowane wagi oceny: poprawność 25%, implementacje 30%, metodologia i analiza 25%, odtwarzalność 10%, dokumentacja i obrona 10%.

Dane i oprogramowanie odniesienia

Podstawą może być generator o kontrolowanych parametrach. Jako uzupełnienie można wykorzystać CIC IDS2017 do analizy ruchu, wybrany zbiór UCI do danych tabelarycznych lub szeregów i przykłady Open3D do chmur punktów. Przed użyciem należy sprawdzić licencję, wersję i wymagane cytowania.

CIC IDS2017: https://www.unb.ca/cic/datasets/ids-2017.html

UCI: https://archive.ics.uci.edu/datasets

Open3D ICP: https://www.open3d.org/docs/release/tutorial/pipelines/icp_registration.html

Tematy ogólne

1 Dyfuzja na siatce z adaptacyjnym podziałem

Identyfikator O01 | Zespół 1–2 osoby

Opis

Projekt przedstawia rozchodzenie się temperatury lub stężenia substancji na dwuwymiarowej siatce. Wartość w komórce zmienia się na podstawie jej stanu i czterech sąsiadów. Należy zaimplementować jawny krok równania dyfuzji, wykorzystując osobne bufory dla stanu bieżącego i następnego. Celem jest zbadanie kosztu obliczeń lokalnych oraz wymiany danych na granicach obszarów.

Program powinien przyjmować rozmiar siatki, liczbę kroków, współczynnik dyfuzji i stan początkowy. Należy ustalić warunki brzegowe i dobrać krok czasowy spełniający warunek stabilności. Wynikiem jest końcowa macierz oraz mapa temperatury; zapis animacji jest opcjonalny.

Zakres implementacji

OpenMP – Podzielić wiersze lub kafle siatki między wątki. Po każdym kroku zsynchronizować wykonanie i zamienić bufory, unikając jednoczesnego odczytu i nadpisywania tego samego stanu.

MPI – Podzielić siatkę na pasy. Każdy proces przechowuje dodatkowe wiersze brzegowe, czyli halo, i wymienia je z sąsiadami przed obliczeniem zależnych komórek.

CUDA lub OpenCL – Przypisać komórki do wątków GPU i porównać odczyty z pamięci globalnej z kaflem przechowywanym w pamięci współdzielonej lub lokalnej. Kolejne kroki wykonywać bez kopiowania całej siatki do CPU.

Dodatkowe wyniki do raportu

Sprawdzić stan stały oraz małą siatkę z ręcznie obliczonym krokiem. Dla co najmniej trzech rozmiarów pokazać czas kroku, przyspieszenie, efektywność CPU/MPI i udział komunikacji. Oddzielić zapis map od obliczeń.

Zakres zespołu

1 osoba: podział na pasy i jeden stencil pięciopunktowy. 2 osoby: dodatkowo kafle 2D albo nakładanie wymiany halo na obliczenia wnętrza obszaru.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy nakładanie wymiany halo na obliczenia przesuwa próg opłacalności MPI? Rozszerzyć na nierównomierny koszt komórek.

Tematy ogólne

2 Kolizje cząstek i struktury przestrzenne

Identyfikator O02 | Zespół 1–2 osoby

Opis

Projekt obejmuje symulację ruchu wielu cząstek na planszy 2D i wykrywanie par, których odległość jest mniejsza od sumy promieni. Metoda sprawdzająca wszystkie pary stanowi referencję. Główna implementacja ma korzystać z regularnej siatki komórek, aby porównywać jedynie obiekty leżące blisko siebie. Obliczenia powinny dawać tę samą listę kolizji niezależnie od podziału pracy.

Program generuje pozycje i prędkości z podanym ziarnem, aktualizuje ruch i odbicia od granic oraz zapisuje liczbę kolizji. W podstawowym zakresie wystarcza wykrywanie kolizji; rozwiązywanie fizycznych zderzeń nie jest wymagane. Należy zapobiec podwójnemu zliczaniu tej samej pary.

Zakres implementacji

OpenMP – Równolegle aktualizować pozycje i obsługiwać komórki. Listy wykrytych par budować lokalnie, a następnie scalać, aby ograniczyć sekcje krytyczne.

MPI – Podzielić planszę na obszary, przekazywać cząstki po przekroczeniu granicy i wymieniać kopie obiektów potrzebnych do testów kolizji przy brzegu.

CUDA lub OpenCL – Wyznaczyć identyfikatory komórek, pogrupować cząstki i równolegle sprawdzać pary w sąsiednich komórkach. Rozmiar sąsiedztwa musi odpowiadać promieniom i wielkości komórki.

Dodatkowe wyniki do raportu

Porównać z referencją wszystkie pary dla małych układów. Zmierzyć czas kroku dla rozkładu równomiernego i skupisk, koszt budowy siatki oraz migracji MPI. Pokazać skalowanie wraz z liczbą cząstek.

Zakres zespołu

1 osoba: jednakowe promienie i detekcja kolizji. 2 osoby: dodatkowo drugi rozmiar lub sposób budowy siatki oraz analiza silnie skupionych układów.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy adaptacyjny rozmiar komórki i równoważenie obciążenia wygrywają ze stałą siatką?

3 Filtracja FIR i próg wyboru algorytmu

Identyfikator O03 | Zespół 1–2 osoby

Opis

Projekt dotyczy filtracji sygnału dźwiękowego filtrem o skończonej odpowiedzi impulsowej FIR. Ten sam wynik należy obliczać przez bezpośrednią konwolucję oraz metodę FFT z podziałem na zachodzące bloki, np. overlap-save. Dla krótkich filtrów istotny jest narzut uruchomienia, a dla długich liczba operacji. Zadaniem jest wyznaczenie granicy opłacalności obu podejść.

Program przyjmuje sygnał WAV lub tablicę próbek, współczynniki filtru, długość bloku i wybraną metodę. Zapisuje sygnał wynikowy oraz czasy. Trzeba jawnie ustalić dopełnienie sygnału i sposób traktowania początku i końca. FFT może pochodzić z biblioteki; własna implementacja obejmuje blokowanie i składanie wyniku.

Zakres implementacji

OpenMP – Przydzielić niezależne bloki sygnału do wątków. Każdy blok musi mieć próbki poprzedzające potrzebne do poprawnego obliczenia konwolucji.

MPI – Rozdzielić segmenty wraz z wymaganym nakładaniem i zebrać wyłącznie poprawne fragmenty wyników. Zmierzyć osobno dystrybucję danych.

CUDA lub OpenCL – Zrealizować bezpośredni FIR albo potok wsadowych FFT i mnożenia widm. Dane oraz współczynniki pozostawiać na urządzeniu pomiędzy etapami.

Dodatkowe wyniki do raportu

Użyć impulsu jednostkowego, sinusoidy i szumu. Pokazać błąd względem referencji, widma przed i po filtracji oraz czas dla kilku długości filtru i bloku. Sprawdzić granice segmentów.

Zakres zespołu

1 osoba: jeden kanał i jeden wariant GPU; obie metody w referencji CPU. 2 osoby: oba warianty GPU i model wyboru metody.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy model długości filtru i partii przewiduje wybór direct lub FFT na nieznanym urządzeniu?

Tematy ogólne

4 Redukcje odporne na błąd numeryczny

Identyfikator O04 | Zespół 1–2 osoby

Opis

Projekt porównuje sposoby równoległego sumowania liczb zmiennoprzecinkowych. Zmiana kolejności dodawania może zmieniać wynik, szczególnie gdy występują liczby o bardzo różnych wartościach lub znoszące się składniki. Należy zestawić zwykłą redukcję drzewiastą z wariantem wykorzystującym sumowanie kompensowane w lokalnych fragmentach. Celem jest jednoczesna ocena szybkości i błędu.

Program generuje lub wczytuje tablicę i oblicza sumę oraz średnią. Udostępnia rozmiar bloku, typ danych i metodę sumowania. Wynik należy porównać z dokładniejszą referencją, np. wielką precyzją dla ograniczonych rozmiarów. Samo użycie float64 nie jest automatycznie dokładnym wzorcem.

Zakres implementacji

OpenMP – Obliczać sumy lokalne w wątkach i łączyć je kontrolowaną redukcją. Zestawić standardową redukcję OpenMP z własnym łączeniem częściowych wyników.

MPI – Rozdzielić dane, policzyć lokalne sumy i zredukować je. Sprawdzić, czy zmiana liczby procesów zmienia błąd oraz końcowe cyfry wyniku.

CUDA lub OpenCL – Wykonać redukcję hierarchiczną wewnątrz bloków i między blokami. Porównać kilka rozmiarów bloku, zachowując wybrany typ akumulacji.

Dodatkowe wyniki do raportu

Uwzględnić dodatnie wartości, mieszaninę skal i znoszenie znaków. Raportować błąd bezwzględny, względny tylko dla niezerowej referencji, przepustowość i powtarzalność bitową. Zysk czasu oceniać wraz z jakością wyniku.

Zakres zespołu

1 osoba: float32 i dwa sposoby sumowania. 2 osoby: dodatkowo float64 albo wariant z ustaloną kolejnością łączenia niezależną od liczby wykonawców.

Rozwinięcie do artykułu lub pracy magisterskiej

Jaki koszt wydajności ma ograniczenie błędu i odtwarzalność wyniku przy zmianie liczby wykonawców?

5 Kompakcja i skan prefiksowy

Identyfikator O05 | Zespół 1–2 osoby

Opis

Projekt polega na usuwaniu z tablicy rekordów niespełniających zadanego warunku, z zachowaniem kolejności pozostałych elementów. Najpierw powstaje maska zer i jedynek, następnie skan prefiksowy wyznacza miejsce każdego zachowanego rekordu. Ostatni etap przenosi elementy do zwartej tablicy. Taki potok jest podstawą wielu operacji analitycznych i algorytmów GPU.

Program przyjmuje tablicę liczb lub prostych rekordów oraz predykat, np. zakres wartości. Zwraca liczbę dopasowań i tablicę wynikową. Musi poprawnie działać, gdy nie zostaje żaden element, zostają wszystkie albo rozmiar danych nie jest wielokrotnością bloku.

Zakres implementacji

OpenMP – Obliczać skany lokalne, następnie offsety bloków i zapis wyników do rozłącznych zakresów. Porównać koszt poszczególnych faz.

MPI – Każdy proces filtruje swój fragment. Operacja Exscan na liczbach dopasowań wyznacza globalny offset, dzięki któremu można złożyć wynik w prawidłowej kolejności.

CUDA lub OpenCL – Zaimplementować skan blokowy i przenoszenie wybranych rekordów. Biblioteczny skan może być punktem odniesienia; wybrany rdzeń potoku powinien być własną implementacją.

Dodatkowe wyniki do raportu

Testować maski losowe oraz długie serie zer i jedynek. Pokazać czas skanu i całej kompakcji przy różnych udziałach zachowanych rekordów, przepustowość oraz zużycie pamięci.

Zakres zespołu

1 osoba: liczby całkowite i jeden predykat. 2 osoby: drugi rozmiar rekordu albo porównanie ze strategią opartą na lokalnych buforach dopasowań.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy selektywność i zagęszczenie maski pozwalają dobrać algorytm kompakcji?

Tematy ogólne

6 Monte Carlo z kontrolą dokładności

Identyfikator O06 | Zespół 1–2 osoby

Opis

Projekt wykorzystuje metodę Monte Carlo do obliczania całki przez losowanie punktów i uśrednianie wartości funkcji. Należy przygotować co najmniej dwie funkcje o znanej całce, aby ocenić błąd estymacji. Głównym celem jest pomiar czasu potrzebnego do uzyskania zadanej dokładności, a także kosztu generowania liczb losowych i agregacji wyników.

Program przyjmuje funkcję, liczbę próbek, ziarno i rozmiar partii. Zwraca estymatę, błąd względem rozwiązania kontrolnego i przedział ufności. Losowania muszą korzystać z rozłącznych strumieni lub jednoznacznego mapowania indeksu próbki na stan generatora.

Zakres implementacji

OpenMP – Podzielić próbki na partie, wyznaczać lokalne sumy i statystyki potrzebne do oszacowania wariancji, a następnie je scalić.

MPI – Przypisać procesom rozłączne zakresy próbek i zredukować statystyki. Unikać uruchamiania identycznego strumienia losowego na wszystkich procesach.

CUDA lub OpenCL – Równolegle generować próbki, obliczać funkcję i redukować wyniki. Osobno oszacować koszt generatora oraz samej funkcji podcałkowej.

Dodatkowe wyniki do raportu

Wykonać eksperymenty dla kilku liczb próbek i wielu ziaren. Pokazać błąd oraz czas do zadanej dokładności, próbki/s i zachowanie przedziałów ufności. Dla porównań dokładności zachować ten sam budżet losowań.

Zakres zespołu

1 osoba: stała liczba próbek i klasyczne losowanie. 2 osoby: dodatkowo próbkowanie warstwowe lub reguła kończenia oparta na dokładności.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak strategie generowania próbek wpływają na czas osiągnięcia dokładności, zamiast tylko próbki/s?

Tematy ogólne

7 Symulacja kolejek wielu scenariuszy

Identyfikator O07 | Zespół 1–2 osoby

Opis

Projekt obejmuje symulację kolejki M/M/c z losowymi czasami między przyjściami klientów i losowymi czasami obsługi. Program śledzi przyjścia, kolejkę oczekujących i zakończenia obsługi przez c stanowisk. Równoległość realizuje się przez niezależne replikacje lub scenariusze. W pojedynczej kolejce trzeba zachować kolejność zdarzeń i ich zależności.

Parametrami są intensywność przyjść, tempo obsługi, liczba stanowisk, długość symulacji i ziarno. Należy uwzględnić okres rozgrzewki. Dla porównań ze stanem ustalonym stosować obciążenie mniejsze od 1. Wynik obejmuje średni czas oczekiwania, długość kolejki i wykorzystanie serwerów.

Zakres implementacji

OpenMP – Przydzielać replikacje do wątków, zapewniając osobne stany generatorów oraz struktur zdarzeń. Porównać statyczny i dynamiczny przydział zadań.

MPI – Rozdzielić scenariusze albo replikacje między procesy i zebrać wyniki do wspólnego zestawienia. Ująć koszt nierównomiernej długości zadań.

CUDA lub OpenCL – Uruchamiać wiele niezależnych replikacji. W zakresie podstawowym przyjąć stałą, niewielką liczbę stanowisk, aby uprościć stan pojedynczej symulacji.

Dodatkowe wyniki do raportu

Porównać średnie i przedziały ufności z wynikiem Erlanga C. Zbadać niskie, średnie i wysokie stabilne obciążenie, czas wykonania oraz replikacje/s. Omówić wpływ rozgrzewki i długości pomiaru.

Zakres zespołu

1 osoba: jedna ustalona liczba stanowisk. 2 osoby: kilka wartości c i grupowanie scenariuszy według przewidywanego kosztu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy grupowanie scenariuszy według kosztu ogranicza nierównomierność i rozbieżność wykonania GPU?

8 Automatyczny wybór CPU lub GPU

Identyfikator O08 | Zespół 1–2 osoby

Opis

Projekt tworzy wspólny mechanizm uruchamiania prostych obliczeń na CPU lub GPU. Należy zaimplementować trzy operacje: przekształcenie elementów map, redukcję sumy i filtrowanie. Program wykonuje krótki zestaw pomiarów kalibracyjnych, a następnie wybiera urządzenie na podstawie rozmiaru zadania. Badanie ma pokazać, kiedy korzyść z GPU pokrywa koszt przesłania danych.

Interfejs przyjmuje operację, rozmiar, parametry danych i tryb automatyczny lub wymuszony. Wyniki wszystkich wariantów muszą być zgodne. Mechanizm wyboru może początkowo korzystać z tabeli progów; bardziej złożony model jest rozszerzeniem. Należy zapisywać czas kalibracji i decyzje.

Zakres implementacji

OpenMP – Wykonać trzy prymitywy przy kontrolowanej liczbie wątków oraz zmierzyć próg opłacalności równoległości względem referencji sekwencyjnej.

MPI – Rozdzielać niezależne zadania pomiarowe lub partie danych. Odróżnić czas jednego zadania od przepustowości całego zbioru zadań.

CUDA lub OpenCL – Przygotować jądra dla operacji i dwa tryby pomiaru: z transferami oraz dla danych już obecnych w pamięci urządzenia.

Dodatkowe wyniki do raportu

Podzielić rozmiary na kalibracyjne i testowe. Raportować trafność wyboru szybszego urządzenia oraz stratę czasu względem najlepszego wariantu dla każdego zadania, uwzględniając koszt strojenia.

Zakres zespołu

1 osoba: progi zależne od rozmiaru na jednym komputerze. 2 osoby: dodatkowo prosty model regresyjny lub walidacja na drugim urządzeniu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy prosty model po niewielu próbach przewiduje szybszy backend? Osobny test na rozmiarach niewidzianych podczas uczenia.

9 Interaktywna analiza widoczności i propagacji światła w chmurach punktów 3D

Identyfikator O09 | Zespół 1–2 osoby

Opis

Projekt dotyczy analizy widoczności i bezpośredniego oświetlenia w dużych chmurach punktów ze skanowania LiDAR lub fotogrametrii. Lokalną geometrię należy dynamicznie odtwarzać jako siatkę trójkątów z wykorzystaniem triangulacji Delaunaya. Trzeba określić sposób rekonstrukcji powierzchni, np. triangulację w lokalnych płaszczyznach, i kryteria odrzucania trójkątów łączących odrębne obiekty. Śledzenie promieni wyznacza obszary widoczne i zacienione; promienie odbite są pomijane.

Użytkownik wskazuje obserwatora lub źródło światła i ogląda wynik w interaktywnej wizualizacji. Przetwarzanie obejmuje aktualny obszar widzenia oraz fragmenty przecinane przez istotne promienie. Należy uwzględnić także przeszkody poza kadrem, które mogą rzucać cień. Program aktualizuje siatkę i wynik po zmianie położenia obserwatora lub światła.

Zakres implementacji

OpenMP – Równolegle rekonstruować lokalne fragmenty siatki i śledzić promienie. Zapewnić spójność geometrii na granicach fragmentów i rozdzielić pomiary rekonstrukcji od analizy widoczności.

MPI – Podzielić chmurę na kafle przestrzenne z halo. Przekazywać promienie między procesami, ustalać najbliższe przecięcie oraz mierzyć komunikację i nierównomierność obciążenia.

CUDA lub OpenCL – Przyspieszyć śledzenie promieni i testy przecięć z trójkątami oraz wybrany etap przygotowania triangulacji. Oddzielić czas aktualizacji geometrii, transferów i obliczeń; wskazać własne jądra oraz udział bibliotek.

Dodatkowe wyniki do raportu

Użyć scen o znanej widoczności, w tym przeszkód poza kadrem i na granicach kafli. Raportować zgodność z referencją, czas generowania siatki, promienie/s, pamięć, FPS i opóźnienie aktualizacji. Zbadać wpływ rozmiaru fragmentu i gęstości chmury; FPS mierzyć przy rzeczywistej aktualizacji wyniku.

Zakres zespołu

1 osoba: lokalna rekonstrukcja, pojedynczy obserwator lub źródło światła i interaktywna wizualizacja. 2 osoby: dodatkowo wiele źródeł światła oraz analiza kosztu aktualizacji przy ich przemieszczaniu.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak dynamiczna rekonstrukcja geometrii wpływa na czas i dokładność analizy widoczności? Porównać z siatką przygotowaną wcześniej; rozszerzyć badania na chmury od milionów do setek milionów punktów, stosownie do dostępnej pamięci.

10 Automatyczna segmentacja budynków z chmur punktów oraz eksport do IFC

Identyfikator O10 | Zespół 1–2 osoby

Opis

Projekt realizuje automatyczną segmentację chmur punktów 3D w celu wykrywania elementów budynku, takich jak ściany, dachy, stropy i słupy. Wykryte segmenty są zamieniane na uproszczone obiekty geometryczne i zapisywane w modelu IFC. Uproszczenie dotyczy geometrii i zakresu informacji; plik wynikowy powinien być zgodny z wybraną wersją formatu IFC.

Program wczytuje jedną lub wiele chmur we wspólnym układzie współrzędnych oraz parametry segmentacji. Zwraca punkty z identyfikatorami segmentów i model IFC z odpowiednimi typami elementów. Należy ustalić jednostki, położenie modelu, tolerancję dopasowania geometrii oraz sposób scalania fragmentów tego samego elementu. Biblioteka może obsługiwać zapis IFC.

Zakres implementacji

OpenMP – Równolegle przetwarzać punkty lub kandydatów na segmenty i dopasowywać geometrię elementów. Kontrolować wspólny zapis etykiet; porównać wydajność dla różnych liczb wątków.

MPI – Podzielić chmurę przestrzennie z obszarami halo. Scalić segmenty na granicach procesów, nadać globalne identyfikatory i usunąć duplikaty elementów. Zmierzyć koszty wymiany danych oraz scalania.

CUDA lub OpenCL – Przyspieszyć wyszukiwanie sąsiadów i wybrany rdzeń segmentacji. Oddzielić czas indeksowania, obliczeń i transferów CPU–GPU; porównywać wyniki dla tych samych parametrów i tolerancji.

Dodatkowe wyniki do raportu

Przetestować prosty budynek, budynek wielokondygnacyjny i chmurę LiDAR. Raportować jakość segmentacji, błąd dopasowania geometrii, liczbę elementów, punkty/s, pamięć i czas etapów. Sprawdzić ponowny odczyt IFC, typy obiektów, jednostki, położenie i brak duplikatów; jakość oceniać względem oznaczonej referencji.

Zakres zespołu

1 osoba: segmentacja ścian, uproszczona rekonstrukcja i eksport do IFC. 2 osoby: dodatkowo wykrywanie dachów i stropów oraz automatyczna klasyfikacja elementów. Wykrywanie słupów może stanowić dalsze rozszerzenie.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy adaptacyjny dobór parametrów segmentacji poprawia jakość i szybkość generowania modeli IFC? Porównać algorytmy pod względem dokładności, czasu i skalowalności oraz sprawdzić przenoszenie parametrów między budynkami o różnej gęstości skanowania.

11 Automatyczne wykrywanie elementów architektonicznych w chmurach punktów z wykorzystaniem AI

Identyfikator O11 | Zespół 1–2 osoby

Opis

Projekt realizuje automatyczne wykrywanie i segmentację elementów architektonicznych w chmurach punktów 3D z wykorzystaniem AI. Rozpoznawane klasy obejmują ściany, okna, drzwi, dachy i elementy ozdobne, zgodnie z zakresem zespołu. Na podstawie segmentów należy generować modele 3D wykrytych obiektów i oceniać zarówno poprawność przypisania klas, jak i jakość odtworzonej geometrii.

Program wczytuje chmurę oraz wybrany model klasyfikacji, oblicza cechy i zapisuje etykiety punktów, segmenty oraz modele 3D. Dopuszczalne jest wykorzystanie wstępnie wytrenowanego modelu, przy opisaniu jego pochodzenia i ograniczeń. Podział danych do uczenia i oceny powinien rozdzielać budynki lub sceny; sąsiednie fragmenty tej samej sceny nie powinny trafiać do obu zbiorów.

Zakres implementacji

OpenMP – Równolegle obliczać cechy, segmentować i klasyfikować punkty lub fragmenty chmury. Kontrolować zagnieżdżenie wątków bibliotek i mierzyć osobno przygotowanie danych oraz predykcję.

MPI – Podzielić chmurę na fragmenty z halo i rozdzielić ich przetwarzanie. Ujednolicić etykiety w częściach wspólnych, scalić segmenty i usunąć duplikaty obiektów na granicach procesów.

CUDA lub OpenCL – Przyspieszyć segmentację, klasyfikację i inferencję modelu AI. Przy użyciu frameworka zaimplementować własne jądro dla istotnego etapu, np. wyznaczania cech lub agregacji sąsiadów. Mierzyć transfery i pełny czas przetwarzania.

Dodatkowe wyniki do raportu

Raportować dokładność klasyfikacji, F1 i IoU dla każdej klasy, jakość modeli 3D, czas, pamięć i punkty/s. Sprawdzić małe oraz słabo reprezentowane elementy, różną gęstość chmury i granice fragmentów. Oceniać geometrię względem oznaczonej referencji oraz podawać koszt przygotowania i inferencji oddzielnie.

Zakres zespołu

1 osoba: wykrywanie ścian, okien i drzwi oraz generowanie modeli 3D ścian. 2 osoby: dodatkowo elementy ozdobne, porównanie modeli AI i automatyczna rekonstrukcja modeli wszystkich wykrywanych klas. Klasy dachów uwzględnić w rozszerzonym zestawie danych.

Rozwinięcie do artykułu lub pracy magisterskiej

Porównać metody geometryczne, PointNet i PointNet++ pod względem jakości segmentacji, rekonstrukcji 3D i skalowalności. Zbadać wpływ wielkości fragmentów oraz halo na jakość i przepustowość dla dużych chmur punktów.

12 AES CTR i potok przetwarzania plików

Identyfikator C01 | Zespół 1–2 osoby

Opis

Projekt dotyczy równoległego szyfrowania plików algorytmem AES w trybie licznikowym CTR. Kolejne bloki strumienia klucza powstają przez szyfrowanie różnych wartości licznika, więc można obliczać je niezależnie. Należy zbadać zarówno samą operację kryptograficzną, jak i potok odczyt, przetwarzanie, zapis. Punktem odniesienia powinna być dojrzała biblioteka korzystająca z dostępnej akceleracji CPU.

Program przyjmuje plik, klucz testowy, wartość początkową licznika i rozmiar bufora. Zapisuje wynik i umożliwia kontrolne odszyfrowanie. Trzeba obsłużyć ostatni niepełny blok oraz zapewnić brak powtórzeń zakresów licznika dla danego klucza. CTR zapewnia poufność, ale sam nie wykrywa modyfikacji danych.

Zakres implementacji

OpenMP – Przydzielić wątkom rozłączne zakresy bloków wraz z odpowiadającymi im licznikami. Oddzielić równoległość obliczeń od buforowania pliku.

MPI – Rozdzielić fragmenty pliku i obliczać licznik na podstawie globalnego numeru bloku. Złożyć dane w pierwotnej kolejności, raportując koszt dystrybucji.

CUDA lub OpenCL – Zaimplementować przetwarzanie niezależnych bloków AES CTR. Porównać wielkości partii i uwzględnić transfery w obu kierunkach.

Dodatkowe wyniki do raportu

Sprawdzić opublikowane wektory testowe AES CTR, zgodność z biblioteką i odzyskanie pliku bajt po bajcie. Raportować GB/s dla jądra oraz całego potoku, wpływ wielkości pliku i bufora.

Zakres zespołu

1 osoba: jeden rozmiar klucza i synchroniczne bufory. 2 osoby: dodatkowo potok z podwójnym buforowaniem i nakładaniem transferów.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy transfery i I/O niwelują przewagę GPU? Zbadać buforowanie i wielkości partii. Zapewnić unikalne zakresy liczników; CTR nie zapewnia uwierzytelnienia.

13 Wyszukiwanie sygnatur w strumieniu

Identyfikator C02 | Zespół 1–2 osoby

Opis

Projekt wyszukuje wiele sygnatur tekstowych w dużym strumieniu bajtów. Sygnatury są literalnymi wzorcami, a wynikiem są ich identyfikatory i pozycje wystąpień. Zastosowaniem jest analiza plików lub zarejestrowanego ruchu pod kątem znanych ciągów. Wymagana jest poprawna obsługa wzorców nakładających się i przechodzących przez granicę fragmentu.

Program wczytuje plik danych i listę wzorców, buduje potrzebne struktury oraz zapisuje dopasowania. Należy rozdzielić czas przygotowania automatu od czasu wyszukiwania. Dla dużej liczby trafień dopuścić tryb zliczania, ale wszystkie porównywane implementacje muszą stosować ten sam tryb.

Zakres implementacji

OpenMP – Podzielić tekst na bloki z zakładką zależną od najdłuższego wzorca. Ustalić właściciela pozycji startowej, aby nie raportować tych samych trafień dwukrotnie.

MPI – Przekazać procesom fragmenty z zakładkami i scalić wyniki z globalnymi pozycjami. Wliczyć koszt przesłania wzorców i tekstu.

CUDA lub OpenCL – Zastosować PFAC lub równoległe dopasowanie rozpoczynane na kolejnych pozycjach. Zbadać koszt częściowych dopasowań oraz rozmiar struktury wzorców.

Dodatkowe wyniki do raportu

Użyć generatora ze znanymi pozycjami trafień, powtarzalnych prefiksów i danych bez dopasowań. Pokazać przepustowość względem długości tekstu, liczby i długości wzorców oraz częstości trafień.

Zakres zespołu

1 osoba: wzorce bajtowe i tryb wsadowy. 2 osoby: drugi układ automatu lub porównanie dwóch sposobów podziału tekstu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy długość wzorców i udział częściowych dopasowań przewidują wybór CPU lub GPU?

14 Ekstrakcja cech przepływów sieciowych

Identyfikator C03 | Zespół 1–2 osoby

Opis

Projekt przygotowuje cechy przepływów sieciowych do późniejszej analizy bezpieczeństwa. Z rekordów pakietów należy utworzyć grupy według klucza obejmującego adresy, porty i protokół, a następnie policzyć liczbę pakietów, sumę długości, czas trwania i wybrane statystyki odstępów czasowych. Szczególnym problemem są nieliczne bardzo duże przepływy oraz liczne przepływy krótkie.

Wejściem jest tabela rekordów pakietów lub wcześniej przekształcony PCAP. Parsowanie PCAP może wykonywać biblioteka. Program zapisuje tabelę cech, stosuje jedną definicję kierunku i końca przepływu oraz określa postępowanie z rekordami poza kolejnością czasową.

Zakres implementacji

OpenMP – Obliczać lokalne agregaty i scalać je według klucza. Dla odstępów czasowych uwzględnić sortowanie i granice fragmentów należących do tego samego przepływu.

MPI – Partycjonować rekordy według skrótu klucza, tak aby kompletny przepływ trafił do jednego procesu. Zmierzyć koszt wymiany rekordów między procesami.

CUDA lub OpenCL – Pracować na kolumnach liczbowych, grupować rekordy i wykonywać redukcje segmentowe. Oddzielić koszt sortowania od obliczania cech.

Dodatkowe wyniki do raportu

Zweryfikować cechy małych przepływów ręcznie. Zmieniać liczbę przepływów, ich długości i skośność rozkładu. Raportować pakiety/s, przepływy/s, pamięć szczytową i czas pełnego przetwarzania.

Zakres zespołu

1 osoba: kierunkowe przepływy i podstawowe agregaty. 2 osoby: dodatkowo statystyki odstępów lub strategia obsługi najdłuższych przepływów.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak rozkład długości przepływów i skośność kluczy wpływają na podział pracy?

15 Korelacja zdarzeń logowania w logach

Identyfikator C04 | Zespół 1–2 osoby

Opis

Projekt analizuje dzienniki logowania i wykrywa serie zdarzeń mogących wskazywać na incydent, np. wiele nieudanych prób dla jednego konta w krótkim czasie. Podstawą jest agregacja w oknach czasowych i jawne reguły alarmowania. Celem obliczeniowym jest przetworzenie dużej liczby zdarzeń przy zachowaniu poprawności na granicach okien i fragmentów danych.

Program wczytuje czas, użytkownika, adres źródłowy i wynik logowania. Parametrami są długość okna i progi reguł. Wynikiem jest lista alarmów z zakresem czasu oraz podsumowanie dla kont. Generator powinien umieszczać oznaczone scenariusze incydentów w zwykłym ruchu.

Zakres implementacji

OpenMP – Rozdzielić konta lub grupy uporządkowanych zdarzeń między wątki. W przypadku podziału czasowego zapewnić dostęp do zdarzeń potrzebnych do okien na brzegu.

MPI – Partycjonować według klucza reguły, np. konta, albo stosować fragmenty czasowe z zakładką. Każdy alarm powinien mieć jednego właściciela.

CUDA lub OpenCL – Po zakodowaniu pól tekstowych wykonywać filtrowanie, sortowanie i zliczanie zdarzeń. Koszt kodowania i przygotowania danych pokazać oddzielnie.

Dodatkowe wyniki do raportu

Sprawdzić zdarzenia dokładnie na granicy okna, puste konta i powtarzające się znaczniki czasu. Raportować precision, recall, zdarzenia/s oraz czas do uzyskania alarmu w odtwarzanym strumieniu.

Zakres zespołu

1 osoba: jedna reguła i jeden rodzaj klucza. 2 osoby: druga reguła oraz porównanie okien stałych i przesuwanych.

Rozwinięcie do artykułu lub pracy magisterskiej

Porównać okna stałe i adaptacyjne przy zmianie intensywności oraz koszt komunikacji.

16 Entropia i segmentacja plików binarnych

Identyfikator C05 | Zespół 1–2 osoby

Opis

Projekt oblicza entropię rozkładu bajtów w kolejnych fragmentach pliku. W każdym oknie powstaje histogram 256 wartości, z którego wyznaczana jest entropia Shannona. Zmiany entropii mogą wskazywać granice obszarów o innej strukturze, np. tekstu, danych skompresowanych lub losowych. Sama wysoka entropia nie jest dowodem złośliwości ani szyfrowania.

Program przyjmuje plik, długość i krok okna. Zwraca wykres entropii względem pozycji oraz listę zmian przekraczających ustalony próg. Należy określić obsługę końcowego niepełnego okna i pominąć składniki histogramu o zerowym prawdopodobieństwie.

Zakres implementacji

OpenMP – Przydzielić okna do wątków i tworzyć osobne histogramy, aby uniknąć współdzielonych liczników przy każdym bajcie.

MPI – Rozdzielić fragmenty pliku z zakładką odpowiadającą oknom. Zachować globalne pozycje i wyeliminować podwójne wyniki na granicach.

CUDA lub OpenCL – Budować histogramy lokalne dla bloków lub okien, następnie obliczać entropię. Zbadać wpływ koncentracji bajtów na konkurencję operacji atomowych.

Dodatkowe wyniki do raportu

Użyć pliku stałych bajtów, kontrolowanego rozkładu i plików z doklejonymi różnymi segmentami. Raportować błąd entropii, GB/s, koszt transferów i dokładność lokalizacji znanych granic.

Zakres zespołu

1 osoba: stałe okna i próg zmiany. 2 osoby: wariant przyrostowej aktualizacji histogramu albo adaptacyjny krok okna.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy adaptacyjny krok okna zmniejsza koszt bez utraty lokalizacji zmian? Sama entropia nie rozstrzyga o złośliwości pliku.

17 Podobieństwo plików na podstawie shingles

Identyfikator C06 | Zespół 1–2 osoby

Opis

Projekt porównuje pliki na podstawie zbiorów krótkich, zachodzących sekwencji bajtów, nazywanych shingles. Dokładną miarą podobieństwa jest Jaccard, czyli stosunek części wspólnej do sumy zbiorów. Dla większych korpusów należy zastosować szkice MinHash, które pozwalają przybliżyć podobieństwo przy mniejszej pamięci. Badanie dotyczy podobieństwa strukturalnego, bez uruchamiania analizowanych plików.

Program przyjmuje katalog, długość sekwencji i rozmiar szkicu. Buduje reprezentacje plików i zwraca ranking podobnych obiektów dla zapytań. Należy zdefiniować obsługę plików krótszych od sekwencji oraz identycznych szkiców i używać wspólnych funkcji haszujących.

Zakres implementacji

OpenMP – Rozdzielić pliki lub fragmenty ekstrakcji między wątki. Przy dzieleniu jednego pliku uwzględnić sekwencje przechodzące przez granice.

MPI – Przechowywać części korpusu w różnych procesach i scalać lokalne rankingi top k albo rozdzielić samo budowanie szkiców.

CUDA lub OpenCL – Równolegle haszować sekwencje i wyznaczać minima tworzące szkic. Porównywać partie szkiców, ograniczając kopiowanie małych obiektów.

Dodatkowe wyniki do raportu

Przygotować warianty plików z wstawieniami, usunięciami i zamianami fragmentów. Zmierzyć recall@k wobec dokładnego rankingu Jaccarda, błąd podobieństwa, czas budowy i czas zapytania.

Zakres zespołu

1 osoba: sekwencje stałej długości i jeden rozmiar korpusu bazowego. 2 osoby: kilka długości sekwencji lub selekcja kandydatów przed porównaniem szkiców.

Rozwinięcie do artykułu lub pracy magisterskiej

Jaki kompromis daje długość szkicu przy różnych rodzajach modyfikacji?

18 Przesiewanie ruchu filtrem Blooma

Identyfikator C07 | Zespół 1–2 osoby

Opis

Projekt implementuje filtr Blooma do szybkiego sprawdzania, czy identyfikator należy do znanego zbioru. Filtr może zwrócić wynik fałszywie dodatni, lecz przy poprawnej realizacji i bez usuwania elementów nie powinien pomijać wstawionych kluczy. Zastosowaniem jest wstępna selekcja rekordów ruchu do dokładniejszej analizy. Należy zbadać kompromis między pamięcią, szybkością i częstością fałszywych trafień.

Program buduje filtr z listy kluczy, a następnie przetwarza zapytania. Parametrami są liczba bitów i funkcji haszujących. Wynik to maska odpowiedzi i statystyki. Dokładny zbiór referencyjny służy do sprawdzania odpowiedzi; koszt jego działania raportuje się osobno.

Zakres implementacji

OpenMP – Podzielić zapytania między wątki. Budowę wykonać przez bezpieczne aktualizacje bitów lub lokalne filtry łączone operacją OR.

MPI – Replikować gotowy filtr w procesach i dzielić zapytania. Zmierzyć czas rozesłania filtru oraz przepustowość po jego udostępnieniu.

CUDA lub OpenCL – Wykonywać wsadowe obliczanie skrótów i odczyty bitów. Dla równoległej budowy zapewnić atomowe ustawianie bitów współdzielonych słów.

Dodatkowe wyniki do raportu

Zmierzyć FPR na kluczach spoza zbioru i potwierdzić brak pominięć kluczy wstawionych. Zmieniać zapełnienie, wielkość partii i udział trafień; raportować zapytania/s oraz pamięć.

Zakres zespołu

1 osoba: klasyczny filtr i statyczny zbiór. 2 osoby: dodatkowo filtr blokowy i porównanie przy zbliżonym FPR.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy filtr blokowy zmniejsza koszt pamięci przy stałym FPR? Rozszerzenie na rozproszone shardy.

19 Inferencja prostego detektora anomalii

Identyfikator C08 | Zespół 1–2 osoby

Opis

Projekt przyspiesza obliczanie oceny anomalii dla gotowych cech ruchu sieciowego. Jako podstawę należy wybrać odległość od najbliższego centroidu albo od k najbliższych przykładów referencyjnych. Im większa odległość, tym bardziej nietypowy rekord. Najważniejsza jest własna implementacja etapu odległości i agregacji, a nie samo wywołanie gotowego klasyfikatora.

Program wczytuje dane referencyjne i testowe, parametry skalowania oraz próg alarmu. Dopasowanie skalowania i progu wykonuje wyłącznie na wydzielonych danych treningowych i walidacyjnych. Zapisuje wyniki punktowe oraz decyzje. Dopuszczalny jest generator albo wybrany podzbiór CIC IDS2017.

Zakres implementacji

OpenMP – Rozdzielić rekordy oceniane między wątki. W przypadku k NN utrzymywać lokalne listy najbliższych sąsiadów zamiast pełnej macierzy odległości.

MPI – Podzielić zapytania z replikacją referencji albo podzielić referencję i scalać lokalne wyniki. Wybrany wariant jasno opisać.

CUDA lub OpenCL – Obliczać odległości kafelkowo i redukować je do minimum lub top k. Uwzględnić koszt przesłania partii i modelu referencyjnego.

Dodatkowe wyniki do raportu

Porównać scoring z referencją sekwencyjną. Raportować PR AUC, recall przy wybranym FPR, rekordy/s i P95 czasu partii. Dane rzeczywiste dzielić według dni lub scenariuszy, kontrolując duplikaty.

Zakres zespołu

1 osoba: jedna metoda oceny anomalii. 2 osoby: druga metoda albo redukcja liczby cech z oceną jakości poza zbiorem strojenia.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy selekcja cech lub przybliżenie odległości zmniejsza opóźnienie przy zachowaniu jakości na innym dniu ruchu?

20 Sumy kryptograficzne i weryfikacja zbioru plików

Identyfikator C09 | Zespół 1–2 osoby

Opis

Projekt tworzy manifest integralności zbioru plików na podstawie SHA 256 i później sprawdza, czy pliki się zmieniły. Równoległość dotyczy niezależnych plików lub komunikatów; kolejne bloki jednego standardowego skrótu SHA 256 mają zależności. Istotnym problemem jest efektywne przetwarzanie wielu krótkich plików razem z nielicznymi dużymi.

Program przyjmuje katalog lub listę plików i zapisuje ścieżki, rozmiary oraz skróty. Tryb kontroli zgłasza pliki zmienione, brakujące i nowe. Należy obsłużyć pusty plik i poprawne dopełnienie SHA 256. Pliki na czas eksperymentu powinny być niezmienne.

Zakres implementacji

OpenMP – Rozdzielać pliki między wątki i porównać statyczny oraz dynamiczny przydział. Każde obliczenie skrótu posiada osobny stan.

MPI – Przydzielać procesom partie plików i scalać manifest. Określić, czy dane są współdzielone, lokalne czy przesyłane, ponieważ wpływa to na czas całości.

CUDA lub OpenCL – Przetwarzać niezależne komunikaty w partiach. Rozważyć grupowanie według długości; zmierzyć koszt pakowania buforów i kopiowania danych.

Dodatkowe wyniki do raportu

Zweryfikować skróty wektorami testowymi i biblioteką. Zmierzyć pliki/s oraz GB/s osobno dla małych i dużych plików, a także czas pełnej weryfikacji. Drzewo skrótów nie jest równoważne SHA 256 całego pliku.

Zakres zespołu

1 osoba: statyczne partie plików. 2 osoby: dynamiczna kolejka lub grupowanie według długości z pomiarem narzutu planowania.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy grupowanie plików według długości zmniejsza rozbieżność GPU i koszt planowania? Nie utożsamiać drzewa skrótów ze standardowym SHA 256 całego pliku.

21 Wykrywanie zmian w strumieniu ruchu sieciowego

Identyfikator C10 | Zespół 1–2 osoby

Opis

Projekt wykrywa zmiany rozkładu cech ruchu sieciowego w kolejnych oknach czasu. Dla wybranej cechy, np. długości pakietu, powstaje histogram porównywany ze stabilnym profilem referencyjnym. Alarm oznacza przekroczenie progu ustalonej miary różnicy. Zmiana rozkładu może mieć różne przyczyny, dlatego projekt ocenia wykrywanie zmian, a nie automatyczne rozpoznanie rodzaju ataku.

Program przyjmuje uporządkowane rekordy, szerokość okna, liczbę przedziałów histogramu i próg. Zwraca przebieg miary oraz znaczniki alarmów. Podstawowy profil referencyjny jest stały i wyznaczany z osobnego fragmentu danych. Wszystkie wersje używają identycznych granic przedziałów.

Zakres implementacji

OpenMP – Przetwarzać okna lub cechy równolegle, budując histogramy lokalne. Etap porównania rozkładów wykonywać po zakończeniu zliczania.

MPI – Rozdzielić okna albo fragmenty z zakładką potrzebną dla okien przesuwanych. Utrzymać prawidłową kolejność wyników w czasie.

CUDA lub OpenCL – Zaimplementować histogramy oraz obliczanie wybranej miary, np. odległości L1 znormalizowanych histogramów. Osobno mierzyć oba etapy.

Dodatkowe wyniki do raportu

Użyć generatora ze zmianami nagłymi i stopniowymi. Raportować opóźnienie detekcji w czasie lub próbkach, fałszywe alarmy na jednostkę czasu, rekordy/s i zależność od szerokości okna.

Zakres zespołu

1 osoba: jedna cecha i stałe okna. 2 osoby: druga cecha lub adaptacyjny profil z analizą, czy aktualizacja nie maskuje incydentu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy adaptacyjne okna ograniczają opóźnienie przy tej samej częstości fałszywych alarmów? Rozszerzyć na ruch z różnych dni.

22 Korelacja podobnych zdarzeń bezpieczeństwa

Identyfikator C11 | Zespół 1–2 osoby

Opis

Projekt łączy podobne zdarzenia bezpieczeństwa na podstawie jawnego wektora cech. Zdarzenie może opisywać typ operacji, źródło, konto i przedział czasu. Dla każdego nowego rekordu należy znaleźć wcześniejsze podobne rekordy lub kandydatów przekraczających próg podobieństwa. Celem jest przyspieszenie etapu korelacji, bez budowania kompletnego systemu zarządzania incydentami.

Program otrzymuje tabelę cech, okno historii i definicję miary odległości. Zapisuje listy dopasowań lub top k. Pola kategoryczne trzeba kodować tak, aby nadane numery nie tworzyły sztucznego porządku. Referencja powinna dokładnie porównywać wszystkie dozwolone pary.

Zakres implementacji

OpenMP – Przydzielić zapytania do wątków i utrzymywać lokalne wyniki. Wspólna baza historii pozostaje niezmienna podczas obliczeń jednej partii.

MPI – Podzielić historię między procesy, obliczać lokalne dopasowania i scalać rankingi. Wykluczyć rekordy późniejsze od analizowanego zdarzenia.

CUDA lub OpenCL – Obliczać odległości dla kafli par i wykonywać filtrowanie lub top k. Nie przechowywać pełnej macierzy dla dużych zbiorów.

Dodatkowe wyniki do raportu

Generator powinien zawierać oznaczone serie powiązanych zdarzeń i tło. Raportować precision, recall, P95, zapytania/s i zużycie pamięci przy rosnącej historii.

Zakres zespołu

1 osoba: jedna miara i stała historia. 2 osoby: tani etap selekcji kandydatów z oceną utraconych dopasowań oraz zysku czasu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy wstępne grupowanie według tanich cech redukuje koszt bez utraty istotnych dopasowań?

23 Analiza grafu komunikacji hostów

Identyfikator C12 | Zespół 1–2 osoby

Opis

Projekt buduje graf komunikacji, w którym wierzchołki odpowiadają hostom, a krawędzie zaobserwowanym połączeniom. Należy obliczać stopnie wierzchołków oraz liczbę wspólnych sąsiadów dla wskazanych par. Metryki opisują strukturę komunikacji i mogą służyć do wskazywania zmian. Szczególną uwagę należy poświęcić hostom o bardzo dużej liczbie sąsiadów.

Program wczytuje rekordy przepływów i tworzy graf prosty według ustalonej definicji kierunku. Podstawowo dopuszcza się graf nieskierowany z usuniętymi duplikatami. Wynikiem są metryki hostów i par oraz zestawienie zmian między dwoma oknami. Samych zmian nie należy utożsamiać z incydentami.

Zakres implementacji

OpenMP – Przydzielić wierzchołki lub pary do wątków i porównywać uporządkowane listy sąsiadów. Uwzględnić różny koszt par o różnych stopniach.

MPI – Rozdzielić krawędzie i agregować stopnie. Dla wspólnych sąsiadów podstawowy wariant może replikować graf i dzielić pary zapytań; koszt replikacji jest częścią pomiaru.

CUDA lub OpenCL – Wykonać obliczenia na listach CSR, czyli zwartej reprezentacji sąsiedztwa. Dobrać podział pracy do długości list i unikać zbędnej materializacji macierzy.

Dodatkowe wyniki do raportu

Sprawdzić małe grafy o znanych metrykach, grafy losowe i grafy z hubami. Raportować czas budowy, pary/s, pamięć oraz zróżnicowanie czasu pracy wykonawców.

Zakres zespołu

1 osoba: statyczny graf i zadane pary. 2 osoby: drugi sposób przydziału par albo porównanie kolejnych okien komunikacji.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy podział według stopnia wierzchołków zmniejsza nierównomierność? Sprawdzić użyteczność metryk do wskazywania zmian wzorca komunikacji.

24 K means z adaptacyjnym przypisaniem

Identyfikator I01 | Zespół 1–2 osoby

Opis

Projekt implementuje grupowanie punktów metodą K-means. W każdej iteracji punkt jest przypisywany do najbliższego centroidu, a następnie centroidy są obliczane ponownie jako średnie punktów w grupach. Zastosowaniem może być analiza danych tabelarycznych albo grupowanie kolorów obrazu. Należy badać zarówno etap odległości, jak i aktualizację centroidów oraz komunikację między procesami.

Program przyjmuje macierz punktów, liczbę klastrów, inicjalizację, limit iteracji i tolerancję. Zapisuje etykiety, centroidy i sumę kwadratów odległości SSE. Trzeba określić sposób obsługi pustego klastra. Porównywane implementacje zaczynają od tych samych centroidów i stosują to samo kryterium zatrzymania.

Zakres implementacji

OpenMP – Rozdzielić punkty między wątki. Sumy współrzędnych i liczebności klastrów gromadzić lokalnie, a następnie redukować, ograniczając konflikty zapisu.

MPI – Każdy proces przechowuje część punktów. W każdej iteracji operacja Allreduce łączy sumy i liczebności potrzebne wszystkim procesom do aktualizacji centroidów.

CUDA lub OpenCL – Obliczać odległości kafelkowo i aktualizować statystyki klastrów przez redukcje. Porównać koszt rozproszonych zapisów i uporządkowanej agregacji.

Dodatkowe wyniki do raportu

Użyć chmur o znanych skupiskach, także nierównych. Pokazać czas iteracji, SSE, liczbę iteracji i udział komunikacji dla różnych liczb punktów, cech i klastrów. Ocenić wpływ błędów zmiennoprzecinkowych na przypisania.

Zakres zespołu

1 osoba: algorytm Lloyda i jeden układ danych. 2 osoby: drugi układ pamięci lub strategia redukcji dla nierównych klastrów.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy liczba cech, K i nierównomierność klastrów wskazują najlepszy układ pamięci i strategię redukcji?

25 Klasyfikacja k NN z podziałem bazy

Identyfikator I02 | Zespół 1–2 osoby

Opis

Projekt realizuje klasyfikację metodą k najbliższych sąsiadów. Dla każdego zapytania należy policzyć odległości do przykładów treningowych, wybrać k najmniejszych i ustalić klasę przez głosowanie. Głównym problemem jest koszt przeszukiwania dużej bazy i sposób rozdzielenia jej między wykonawców. Projekt obejmuje dokładne wyszukiwanie, co pozwala jednoznacznie ocenić poprawność.

Program wczytuje wektory cech i etykiety, umożliwia wybór k oraz wielkości partii zapytań. Zapisuje predykcje i identyfikatory sąsiadów. Należy zdefiniować rozstrzyganie remisów. Skalowanie cech dopasowuje się wyłącznie na zbiorze treningowym, a ocenę jakości wykonuje na odrębnym zbiorze.

Zakres implementacji

OpenMP – Podzielić zapytania między wątki i wyznaczać lokalne listy top k. Unikać zapisywania całej macierzy odległości, jeśli nie jest potrzebna.

MPI – Podzielić bazę na fragmenty, rozsyłać zapytania i scalać k kandydatów z każdego procesu do globalnej listy. Dla porównania można replikować bazę i dzielić zapytania.

CUDA lub OpenCL – Obliczać odległości na kaflach zapytań i bazy, następnie wykonywać selekcję top k. Uwzględnić pamięć tymczasową i przesyłanie partii.

Dodatkowe wyniki do raportu

Testować identyczne wektory, remisy i k bliskie rozmiarowi bazy. Raportować zgodność sąsiadów, accuracy, zapytania/s i P95 przy różnych liczbach cech oraz rozmiarach bazy.

Zakres zespołu

1 osoba: odległość euklidesowa i jeden podział MPI. 2 osoby: drugi podział MPI oraz eksperyment wskazujący próg jego opłacalności.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy dzielenie bazy jest lepsze od replikacji i dzielenia zapytań?

26 Dopasowanie szeregów metodą DTW

Identyfikator I03 | Zespół 1–2 osoby

Opis

Projekt porównuje szeregi czasowe metodą dynamicznego dopasowania czasu DTW. Algorytm pozwala zestawić przebiegi o podobnym kształcie, lecz rozciągnięte lub przesunięte w czasie. Wymaga obliczenia tablicy kosztów z zależnościami między sąsiednimi komórkami. Zadaniem programu jest wyznaczanie odległości oraz wyszukiwanie szeregu najlepiej dopasowanego do zapytania.

Program przyjmuje zbiór szeregów, zapytanie i szerokość pasma ograniczającego dopasowanie. Należy jawnie ustalić lokalny koszt, warunki brzegowe i normalizację. W zakresie podstawowym wystarcza wynik odległości; odtworzenie pełnej ścieżki jest opcjonalne. Referencja stosuje takie samo pasmo.

Zakres implementacji

OpenMP – Rozdzielić niezależne pary szeregów między wątki. Każdy wątek posiada bufory kolejnych wierszy tablicy kosztów.

MPI – Podzielić kandydatów między procesy i po obliczeniach zredukować najlepszą odległość wraz z identyfikatorem. Uwzględnić różne długości szeregów.

CUDA lub OpenCL – Obliczać komórki wzdłuż kolejnych przekątnych zgodnie z zależnościami albo przetwarzać wiele krótkich par jednocześnie. Nie zakładać niezależności wszystkich komórek.

Dodatkowe wyniki do raportu

Użyć identycznych przebiegów, znanych przesunięć i szeregów różnej długości. Raportować zgodność odległości, pary/s, pamięć i wpływ szerokości pasma. Przy odrzucaniu kandydatów sprawdzić zachowanie poprawnego najbliższego wyniku.

Zakres zespołu

1 osoba: stała długość i wybrane pasmo. 2 osoby: zmienne długości oraz dolne ograniczenie kosztu eliminujące część kandydatów.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy dolne ograniczenia i grupowanie według długości przyspieszają wyszukiwanie bez utraty dokładności?

27 Potok obrazu dla nakładek AR

Identyfikator R01 | Zespół 1–2 osoby

Opis

Projekt przygotowuje potok przetwarzania obrazu, którego wynik można wykorzystać jako nakładkę w aplikacji rozszerzonej rzeczywistości. Kolejne etapy to rozmycie Gaussa, detekcja krawędzi operatorem Sobela i progowanie maski. Należy zapewnić zgodny wynik trzech implementacji oraz zbadać, czy łączenie etapów i ograniczenie transferów poprawia opóźnienie.

Program wczytuje zapisane wideo lub sekwencję obrazów, przyjmuje parametry filtrów i progu, a następnie zapisuje maski lub klatki z nakładką. Należy ustalić regułę obsługi brzegów obrazu. Dekodowanie i podgląd mogą korzystać z bibliotek. Integracja z goglami nie jest częścią zakresu podstawowego.

Zakres implementacji

OpenMP – Przetwarzać wiersze lub kafle obrazu, zachowując synchronizację między etapami. Dla rozmycia można zastosować separację filtru na dwa przebiegi jednowymiarowe.

MPI – Rozdzielić niezależne klatki albo klipy, a wyniki uporządkować według numeru. Ten wariant mierzy przyspieszenie przetwarzania wsadu, nie pojedynczej klatki.

CUDA lub OpenCL – Wykonać filtry i progowanie na urządzeniu, pozostawiając tam bufory między etapami. Porównać osobne jądra z połączeniem wybranych operacji.

Dodatkowe wyniki do raportu

Sprawdzić obrazy stałe, pojedyncze krawędzie i naturalne nagrania. Raportować zgodność pikseli lub błąd w tolerancji, FPS obliczeń i całego potoku oraz P95 opóźnienia klatki. Uwzględnić różne rozdzielczości.

Zakres zespołu

1 osoba: obrazy w skali szarości i zapisane wideo. 2 osoby: fuzja jąder albo nakładanie transferów z przetwarzaniem kolejnej klatki.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy fuzja etapów i utrzymanie danych na GPU zmniejszają P95 przy tej samej jakości? MPI bada tryb wsadowy.

28 Rejestracja chmur punktów ICP

Identyfikator R02 | Zespół 1–2 osoby

Opis

Projekt dopasowuje dwie chmury punktów 3D metodą ICP w wariancie point-to-point. W każdej iteracji wyszukuje się korespondencje, oblicza transformację sztywną i aktualizuje położenie punktów. Algorytm wymaga przybliżonego ustawienia początkowego; nie jest pełnym systemem lokalizacji i mapowania. Dominującym etapem projektu jest znajdowanie odpowiadających sobie punktów.

Program wczytuje dwie chmury, transformację początkową, próg odległości korespondencji i limit iteracji. Zapisuje końcową macierz transformacji, błąd dopasowania i liczbę iteracji. Rozkład małej macierzy używany do wyznaczania obrotu może wykonywać biblioteka. Należy odrzucać przypadki z niedostateczną liczbą korespondencji.

Zakres implementacji

OpenMP – Równolegle wyszukiwać sąsiadów dla punktów źródłowych i obliczać częściowe statystyki potrzebne do estymacji transformacji.

MPI – Podzielić punkty źródłowe, replikować chmurę docelową i redukować statystyki. Każdy proces powinien w kolejnej iteracji stosować tę samą transformację.

CUDA lub OpenCL – Obliczać korespondencje i redukcje na GPU. W podstawowym wariancie można użyć dokładnego przeglądu par dla ograniczonego rozmiaru chmury.

Dodatkowe wyniki do raportu

Użyć chmur z narzuconym znanym obrotem i przesunięciem, następnie dodać szum. Raportować błąd transformacji, RMSE, liczbę iteracji i czas poszczególnych etapów. Zbadać wpływ inicjalizacji i liczby punktów.

Zakres zespołu

1 osoba: małe chmury, dokładne korespondencje i bliska inicjalizacja. 2 osoby: podpróbkowanie lub struktura przyspieszająca wyszukiwanie. Zalecana realizacja dwuosobowa.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy podpróbkowanie adaptacyjne skraca czas przy zadanym błędzie i różnych początkowych przesunięciach?

29 Filtrowanie chmury punktów i wokselizacja

Identyfikator R03 | Zespół 1–2 osoby

Opis

Projekt zmniejsza liczbę punktów w chmurze 3D przez podział przestrzeni na woksele, czyli regularne komórki. Punkty z jednej komórki zastępowane są centroidem. Taka reprezentacja może przyspieszać kolejne etapy rekonstrukcji i wizualizacji. Należy zbadać koszt przypisywania punktów do komórek, grupowania i obliczania centroidów.

Program przyjmuje chmurę i rozmiar woksela, zapisuje chmurę uproszczoną oraz liczebności komórek. Należy zdefiniować początek siatki i indeksowanie również dla współrzędnych ujemnych. Opcjonalny próg minimalnej liczebności usuwa rzadkie komórki; wszystkie wersje stosują tę samą regułę.

Zakres implementacji

OpenMP – Wyznaczać klucze wokseli i lokalne sumy współrzędnych oraz liczebności. Scalić agregaty, dbając o poprawność komórek występujących w kilku fragmentach.

MPI – Przesłać punkty lub częściowe agregaty do właścicieli kluczy. Każdy woksel musi uzyskać jeden centroid uwzględniający wszystkie należące do niego punkty.

CUDA lub OpenCL – Obliczyć klucze, posortować lub zgrupować punkty i wykonać redukcje segmentowe. Mierzyć czas budowy reprezentacji osobno od zapisu pliku.

Dodatkowe wyniki do raportu

Sprawdzić punkty na granicach komórek, duplikaty i pojedyncze woksele. Raportować punkty/s, pamięć, stopień redukcji oraz błąd odległości względem chmury wejściowej. Uwzględnić skupiska o różnej gęstości.

Zakres zespołu

1 osoba: jeden sposób grupowania i same współrzędne. 2 osoby: porównanie sortowania z haszowaniem albo agregacja dodatkowej cechy, np. koloru.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy haszowanie wygrywa z sortowaniem przy różnym zagęszczeniu punktów?

Data Science

30 Statystyki strumieniowe dużych tabel

Identyfikator D01 | Zespół 1–2 osoby

Opis

Projekt wyznacza podstawowe statystyki dużej tabeli bez konieczności przechowywania całości danych w pamięci. Należy obliczać średnią, wariancję i wybrane korelacje przez łączenie statystyk częściowych. Celem jest porównanie przepustowości przetwarzania blokowego i wpływu kolejności agregacji na dokładność, szczególnie dla danych o dużej średniej i małej wariancji.

Program przyjmuje plik CSV lub binarny, wskazane kolumny i rozmiar bloku. Zapisuje liczbę obserwacji i statystyki. Trzeba jednoznacznie ustalić postępowanie z brakami danych oraz wybór wariancji populacyjnej lub z próby. Referencja powinna korzystać ze stabilnego algorytmu, np. łączalnego wariantu Welforda.

Zakres implementacji

OpenMP – Przetwarzać bloki w wątkach i scalać liczebności, średnie oraz sumy kwadratów odchyleń. Dla korelacji utrzymywać właściwe statystyki wspólne.

MPI – Rozdzielić bloki między procesy i połączyć statystyki tą samą matematycznie poprawną operacją, zamiast uśredniać średnie bez uwzględniania liczebności.

CUDA lub OpenCL – Obliczać statystyki bloków i redukować je hierarchicznie. Zbadać układ kolumnowy oraz koszt przenoszenia kolejnych porcji danych.

Dodatkowe wyniki do raportu

Testować nierówne długości bloków, kolumnę stałą i dane o różnych skalach. Raportować błąd, rekordy/s, pamięć oraz osobne czasy parsowania i agregacji. Dla korelacji kolumn stałych określić wynik niezdefiniowany.

Zakres zespołu

1 osoba: średnia, wariancja i jedna para kolumn. 2 osoby: więcej korelacji albo potok nakładający odczyt na obliczenia.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak wielkość bloku i reprezentacja kolumn wpływają na czas pełnego przetwarzania?

Data Science

31 PageRank grafów o nierównych stopniach

Identyfikator D02 | Zespół 1–2 osoby

Opis

Projekt oblicza ranking wierzchołków grafu skierowanego algorytmem PageRank. Każda iteracja rozdziela wartość wierzchołka między jego sąsiadów i uwzględnia współczynnik tłumienia. Należy obsłużyć wierzchołki bez krawędzi wychodzących oraz zatrzymywać obliczenia po osiągnięciu ustalonej zbieżności. Głównym zagadnieniem jest nierównomierny koszt pracy w grafach z hubami.

Program wczytuje listę krawędzi, tworzy reprezentację rzadką, np. CSR, i zapisuje ranking. Parametrami są współczynnik tłumienia, tolerancja i limit iteracji. Wszystkie warianty stosują ten sam wektor początkowy oraz kryterium zbieżności. Czas konwersji grafu mierzy się oddzielnie.

Zakres implementacji

OpenMP – Rozdzielić wierzchołki lub zakresy krawędzi i wykonywać aktualizacje bez wyścigów. Można użyć wariantu pull z listami krawędzi przychodzących.

MPI – Przydzielić procesom fragmenty grafu i wymieniać potrzebne wartości rankingu lub wkłady. Globalnie redukować masę wierzchołków wiszących i miarę zbieżności.

CUDA lub OpenCL – Wykonać mnożenie rzadkiej macierzy przez wektor i aktualizację rankingu. Dobrać liczbę wątków obsługujących wierzchołek do liczby jego krawędzi.

Dodatkowe wyniki do raportu

Sprawdzić małe grafy i sumę rankingu bliską 1. Raportować residuum, liczbę iteracji, krawędzie/s, czas komunikacji oraz skalowanie dla grafów równomiernych i o nierównych stopniach.

Zakres zespołu

1 osoba: jeden format rzadki i jeden podział grafu. 2 osoby: porównanie podziału według wierzchołków i liczby krawędzi.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy podział według liczby krawędzi ogranicza nierównomierność względem podziału według wierzchołków?

Data Science

32 Wyszukiwanie wektorowe i top k

Identyfikator D03 | Zespół 1–2 osoby

Opis

Projekt realizuje wyszukiwanie podobnych wektorów, np. gotowych reprezentacji tekstów lub obrazów. Dla partii zapytań należy znaleźć k najbardziej podobnych elementów bazy. Zakres podstawowy obejmuje dokładne wyszukiwanie, a wektory są generowane lub wczytywane z pliku. Trenowanie modelu tworzącego reprezentacje nie jest wymagane.

Program przyjmuje bazę, zapytania, k i wielkość partii. Zwraca identyfikatory oraz odległości. Należy wybrać odległość euklidesową albo podobieństwo cosinusowe i zdefiniować obsługę remisów. Przy podobieństwie cosinusowym trzeba ustalić postępowanie z wektorami zerowymi.

Zakres implementacji

OpenMP – Rozdzielić zapytania, obliczać odległości w blokach i utrzymywać top k. Zoptymalizowane operacje biblioteczne wykorzystać jako dodatkową referencję wydajności.

MPI – Podzielić bazę między procesy, wysłać zapytania i scalić lokalne top k do odpowiedzi globalnej. Wymiana nie powinna obejmować całej macierzy odległości.

CUDA lub OpenCL – Zastosować kafelkowanie i selekcję wyników na urządzeniu. Oddzielić czas budowania lub przesyłania bazy od czasu kolejnych zapytań.

Dodatkowe wyniki do raportu

Zbadać rozmiar bazy, liczbę wymiarów, k i wielkość partii. Raportować zgodność rankingu, zapytania/s, P95 i pamięć szczytową. Przy aproksymacji mierzyć recall@k względem dokładnej referencji.

Zakres zespołu

1 osoba: jedna metryka i float32. 2 osoby: dodatkowo niższa precyzja lub kompresja z porównaniem czasu przy ustalonej jakości rankingu.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy obniżenie precyzji lub kompresja zmniejsza koszt przy akceptowalnym recall@k?

33 Złączenie tabel i skośne klucze

Identyfikator D04 | Zespół 1–2 osoby

Opis

Projekt implementuje złączenie wewnętrzne dwóch tabel po kluczu. Każda para wierszy o identycznym kluczu musi znaleźć się w wyniku, również gdy klucze powtarzają się po obu stronach. Należy zbadać wpływ skośnego rozkładu, w którym niewielka liczba kluczy występuje bardzo często. Taki przypadek powoduje nierównomierne obciążenie oraz duży wynik.

Program wczytuje lub generuje tabele z kluczem całkowitym i identyfikatorem wiersza. Zapisuje pary identyfikatorów lub ich deterministyczne podsumowanie kontrolne. Pełne wyniki należy sprawdzać na małych danych. W pomiarach jasno odróżnić samo zliczanie par od materializacji wyniku.

Zakres implementacji

OpenMP – Podzielić tabele na partycje i wykonać hash join lub sort-merge join. Przydzielić rozłączne zakresy wyniku albo zastosować lokalne bufory.

MPI – Partycjonować obie tabele tą samą funkcją klucza i wymienić rekordy między procesami. Następnie wykonać lokalne złączenia, mierząc koszt shuffle.

CUDA lub OpenCL – Zaimplementować wybrany etap złączenia, np. dopasowanie posortowanych grup i generowanie par. Zapewnić obsługę duplikatów i ograniczeń pamięci.

Dodatkowe wyniki do raportu

Użyć kluczy unikalnych, duplikatów, braku dopasowań i gorących kluczy. Raportować czas, wielkość wyniku, pamięć i nierównomierność obciążenia. Porównania wykonywać dla tej samej semantyki złączenia.

Zakres zespołu

1 osoba: jedna metoda i klucze całkowite. 2 osoby: specjalna obsługa gorących kluczy lub drugi algorytm złączenia.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy specjalna obsługa gorących kluczy redukuje nierównomierność i pamięć szczytową?

Data Science

34 Podobieństwo zbiorów i szkice danych

Identyfikator D05 | Zespół 1–2 osoby

Opis

Projekt porównuje dokumenty lub inne obiekty opisane zbiorami elementów. Należy obliczać dokładne podobieństwo Jaccarda oraz przybliżenie na podstawie szkiców MinHash. W przeciwieństwie do częstości słów reprezentacja zbiorowa uwzględnia obecność tokenu, a nie liczbę jego wystąpień. Celem jest ocena, ile czasu i pamięci można oszczędzić kosztem kontrolowanego błędu.

Program wczytuje zbiory tokenów lub proste teksty, usuwa duplikaty i buduje szkice. Zapisuje podobieństwa dla zadanych par albo ranking względem zapytania. Należy stosować identyczne haszowanie we wszystkich wariantach oraz określić konwencję podobieństwa zbiorów pustych.

Zakres implementacji

OpenMP – Przydzielić dokumenty lub pary do wątków. Dla dokładnej referencji porównywać uporządkowane zbiory, a dla szkiców liczyć zgodne pozycje.

MPI – Rozdzielić pary porównań albo fragmenty korpusu i zebrać wyniki. Koszt replikacji szkiców lub przesyłania zapytań uwzględnić oddzielnie.

CUDA lub OpenCL – Równolegle wyznaczać wartości skrótów, minima oraz podobieństwa szkiców. Grupować zadania o podobnej długości, jeśli pozwala to ograniczyć nieaktywną pracę.

Dodatkowe wyniki do raportu

Generator powinien kontrolować rozmiary zbiorów i ich część wspólną. Raportować błąd estymacji, zgodność rankingu, pary/s, czas budowy szkiców i pamięć dla kilku długości szkicu.

Zakres zespołu

1 osoba: porównania zadanej listy par. 2 osoby: wyszukiwanie top k lub strategia grupowania zbiorów o różnych rozmiarach.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak długość szkicu wpływa na jakość rankingu i czas dla różnych rozkładów rozmiaru zbiorów?

35 Bootstrap przedziałów ufności

Identyfikator D06 | Zespół 1–2 osoby

Opis

Projekt wyznacza przedziały ufności metodą bootstrap. Z danych wielokrotnie losuje się ze zwracaniem próbki o tym samym rozmiarze i oblicza wybraną statystykę, np. średnią. Rozkład wyników replikacji służy do wyznaczenia przedziału percentylowego. Replikacje są niezależne, dzięki czemu naturalnie nadają się do równoległości.

Program przyjmuje próbę danych, liczbę replikacji, poziom ufności i ziarno. Zwraca estymatę, granice przedziału i statystyki czasu. Losowania muszą być niezależne między wykonawcami. Należy ustalić metodę obliczania kwantyli, aby różnice bibliotek nie zmieniały porównania.

Zakres implementacji

OpenMP – Rozdzielić replikacje między wątki. Każda replikacja generuje indeksy i oblicza statystykę bez konieczności przechowywania całej kopii próby.

MPI – Przydzielić procesom replikacje, a następnie zebrać statystyki do wyznaczenia kwantyli. Osobno mierzyć koszt udostępnienia danych wejściowych.

CUDA lub OpenCL – Równolegle generować indeksy i redukować wylosowane wartości. Ograniczyć rozmiar buforów, przetwarzając replikacje partiami.

Dodatkowe wyniki do raportu

Porównać obliczenia dla tych samych indeksów resamplingu. Pokrycie przedziałów oceniać na wielu niezależnie wygenerowanych zbiorach danych, nie tylko na wielu replikacjach jednej próby. Raportować szerokość, pokrycie i czas.

Zakres zespołu

1 osoba: bootstrap średniej. 2 osoby: druga statystyka albo adaptacyjne ograniczanie liczby replikacji z kontrolą pokrycia.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy ograniczenie liczby replikacji według stabilności wyniku daje zysk przy zachowaniu pokrycia?

36 Wydobywanie częstych zbiorów elementów

Identyfikator D07 | Zespół 1–2 osoby

Opis

Projekt wyszukuje często współwystępujące elementy w zbiorze transakcji, np. produktów w koszykach. Zakres podstawowy obejmuje liczności pojedynczych elementów i par, bez implementacji pełnego algorytmu Apriori. Dla pary należy obliczyć support, czyli odsetek transakcji zawierających oba elementy. Koszt zależy od liczby elementów, długości transakcji i gęstości danych.

Program wczytuje transakcje, usuwa powtórzenia elementów wewnątrz jednej transakcji i przyjmuje próg support. Zapisuje częste elementy oraz pary z licznościami. Dozwolone są listy wystąpień lub bitmapy, gdzie bit oznacza obecność elementu w danej transakcji. Należy kontrolować rozmiar zbioru kandydatów.

Zakres implementacji

OpenMP – Zliczać pary z transakcji w lokalnych strukturach albo przydzielać pary elementów do wątków i przecinać ich listy wystąpień.

MPI – Rozdzielić transakcje, wyznaczać częściowe zliczenia i redukować je dla wspólnych identyfikatorów par. Alternatywnie dzielić kandydatów przy replikowanych danych.

CUDA lub OpenCL – Dla reprezentacji bitmapowej przecinać słowa bitowe i zliczać ustawione bity operacją popcount. Mierzyć także koszt budowy bitmap.

Dodatkowe wyniki do raportu

Sprawdzić ręcznie mały zbiór koszyków. Zmieniać średnią długość transakcji, rozmiar słownika i próg support. Raportować czas przygotowania, pary/s, pamięć i zgodność zliczeń.

Zakres zespołu

1 osoba: ograniczony słownik i pary elementów. 2 osoby: porównanie bitmap z listami wystąpień. Pełne wydobywanie dłuższych zbiorów jest rozszerzeniem badawczym.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy reprezentacja bitmapowa wygrywa z listami przy zmiennej gęstości? Magisterka może rozszerzyć na dłuższe zbiory.

Data Science

37 Sortowanie jako etap analizy danych

Identyfikator D08 | Zespół 1–2 osoby

Opis

Projekt porównuje sortowanie rekordów jako etap przygotowania do grupowania danych. Po uporządkowaniu po kluczu należy policzyć liczności lub sumy wartości w każdej grupie. Dzięki temu można ocenić, czy najszybszy sort sam w sobie daje również najszybszy cały potok. Poszczególne technologie mogą używać różnych algorytmów, ale muszą realizować tę samą semantykę.

Program generuje rekordy z kluczem i wartością, wybiera backend oraz zapisuje wynik grupowania. Należy ustalić, czy wymagana jest stabilność sortowania; jeśli tak, zachować kolejność identyfikatorów w obrębie równych kluczy. Wyniki kontrolować na małych tablicach i przez sprawdzenie porządku.

Zakres implementacji

OpenMP – Zaimplementować sortowanie przez scalanie z równoległym podziałem i ograniczeniem zbyt małych zadań. Grupowanie wykonać z obsługą grup na granicach bloków.

MPI – Zastosować sample sort: wybrać podziały kluczy na podstawie próbki, wymienić partycje i posortować lokalnie. Scalić agregaty grup przecinających granice procesów.

CUDA lub OpenCL – Zaimplementować wybrany rdzeń radix sort lub własne grupowanie po sortowaniu bibliotecznym. Jasno wskazać wkład własny i koszt każdego etapu.

Dodatkowe wyniki do raportu

Użyć danych losowych, prawie uporządkowanych i z licznymi duplikatami. Raportować czas sortowania, grupowania i całości, rekordy/s, komunikację oraz pamięć szczytową.

Zakres zespołu

1 osoba: klucze całkowite i jeden algorytm na backend. 2 osoby: drugi sposób próbkowania podziałów lub stabilny wariant złożonego klucza.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy próbkowanie rozkładu kluczy pozwala dobrać partycje i zmniejszyć koszt końcowego group by?

38 Regresja logistyczna z redukcją gradientów

Identyfikator D09 | Zespół 1–2 osoby

Opis

Projekt implementuje uczenie binarnej regresji logistycznej metodą mini-batch SGD. Dla partii przykładów obliczane są predykcje, funkcja straty i gradient, a następnie aktualizowane wagi. Obliczenia na przykładach są równoległe, lecz wszystkie gradienty danej aktualizacji muszą zostać właściwie połączone. Celem jest ocena czasu osiągnięcia określonej jakości modelu.

Program wczytuje cechy i etykiety, przyjmuje współczynnik uczenia, wielkość partii, liczbę epok i ziarno. Zapisuje model i historię straty. Należy stosować stabilne numerycznie obliczenie sigmoidy i straty, a skalowanie cech dopasować wyłącznie na zbiorze treningowym.

Zakres implementacji

OpenMP – Obliczać gradienty lokalne dla części przykładów i redukować je przed jedną wspólną aktualizacją wag. Unikać jednoczesnych niesynchronizowanych zmian parametrów.

MPI – Rozdzielić przykłady należące do globalnej partii między procesy. Operacja Allreduce łączy gradienty, po czym wszystkie procesy wykonują tę samą aktualizację.

CUDA lub OpenCL – Zaimplementować obliczanie predykcji, gradientu i aktualizacji. Ograniczyć transfery, utrzymując parametry i w miarę możliwości dane na urządzeniu.

Dodatkowe wyniki do raportu

Sprawdzić gradient na małym przykładzie, np. różnicami skończonymi. Raportować stratę, accuracy, czas epoki i czas do zadanego progu jakości. Przy skalowaniu zachować globalny rozmiar partii albo wyjaśnić jego zmianę.

Zakres zespołu

1 osoba: SGD bez dodatkowych optymalizatorów. 2 osoby: analiza wielkości partii lub rzadziej synchronizowany wariant z kontrolą zmiany zbieżności.

Rozwinięcie do artykułu lub pracy magisterskiej

Jak wielkość partii i częstotliwość synchronizacji wpływają na czas do jakości?

39 Przeszukiwanie hiperparametrów z planowaniem zadań

Identyfikator D10 | Zespół 1–2 osoby

Opis

Projekt realizuje przeszukiwanie hiperparametrów niewielkiego modelu, np. regresji logistycznej. Zbiór konfiguracji ma być taki sam w trzech implementacjach, a zadania mogą różnić się kosztem. Należy zbadać, jak sposób przydzielania konfiguracji wpływa na czas ukończenia całego eksperymentu. Ważna jest odtwarzalność wyników i nieprzekraczanie ustalonego budżetu.

Program przyjmuje listę konfiguracji, ziarna i podział danych. Zapisuje wynik walidacji, czas oraz status każdej próby, a następnie wskazuje najlepszą konfigurację. Zbiór testowy pozostaje odłożony do końcowej oceny. Zakres nie obejmuje dużych sieci ani pełnego systemu AutoML.

Zakres implementacji

OpenMP – Wykonywać konfiguracje jako niezależne zadania. Kontrolować liczbę wątków wewnątrz modelu, aby uniknąć nadmiarowego zagnieżdżania równoległości.

MPI – Zastosować kolejkę zadań z procesem zarządzającym albo statyczny podział konfiguracji. Rejestrować chwile rozpoczęcia i zakończenia prób.

CUDA lub OpenCL – Przygotować własny wsadowy etap gradientu lub scoringu dla kilku małych modeli. Samo uruchamianie gotowej biblioteki na GPU nie wyczerpuje wkładu programistycznego.

Dodatkowe wyniki do raportu

Porównać kompletność i zgodność rezultatów przy tym samym budżecie. Raportować czas całej serii, wykorzystanie wykonawców, narzut planisty i rozkład czasów prób. Uwzględnić konfiguracje o nierównym koszcie.

Zakres zespołu

1 osoba: mała siatka i statyczny podział zadań. 2 osoby: dynamiczna kolejka albo planowanie według przewidywanego czasu konfiguracji.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy przewidywanie czasu konfiguracji skraca całe przeszukiwanie? Weryfikować na konfiguracjach niewykorzystanych do strojenia planisty.

Data Science

40 Macierz kowariancji i przygotowanie do PCA

Identyfikator D11 | Zespół 1–2 osoby

Opis

Projekt oblicza macierz kowariancji dużej tabeli jako etap przygotowania do analizy głównych składowych PCA. Najpierw trzeba wyznaczyć średnie cech, a następnie iloczyny odchyleń i ich sumy. Liczba elementów macierzy rośnie kwadratowo z liczbą cech, dlatego istotne są blokowanie, pamięć i komunikacja. Rozkład własny małej macierzy może zostać wykonany przez bibliotekę.

Program przyjmuje macierz danych, rozmiar bloku i typ obliczeń. Zapisuje średnie oraz macierz kowariancji z jawnie wybraną normalizacją. Dla danych kompletnych wszystkie pary cech wykorzystują te same obserwacje. Obsługa braków może zostać ograniczona do odrzucenia wierszy.

Zakres implementacji

OpenMP – Obliczyć średnie redukcją, a następnie rozdzielić bloki par kolumn. Wykorzystać symetrię macierzy i unikać wielokrotnego zapisu tych samych elementów.

MPI – Podzielić rekordy, wyznaczyć globalne średnie i zredukować częściowe macierze iloczynów odchyleń. Nie uśredniać lokalnych kowariancji bez poprawki na różnice średnich.

CUDA lub OpenCL – Przetwarzać bloki wycentrowanych danych, wykonując iloczyny i redukcje. Przynajmniej jeden dominujący etap powinien mieć własne jądro; biblioteka stanowi referencję.

Dodatkowe wyniki do raportu

Użyć danych z zadanym spektrum oraz o różnych skalach cech. Raportować błąd macierzy, symetrię, czas, pamięć i udział komunikacji. Przy PCA porównywać podprzestrzenie zamiast znaków wektorów własnych.

Zakres zespołu

1 osoba: kompletne dane i float32 lub float64. 2 osoby: drugi typ precyzji albo obliczenia blokowe dla danych większych od pamięci GPU.

Rozwinięcie do artykułu lub pracy magisterskiej

Kiedy obliczenia blokowe lub niższa precyzja zmniejszają koszt przy zachowaniu jakości głównych składowych?

41 Agregacje w oknach szeregów czasowych

Identyfikator D12 | Zespół 1–2 osoby

Opis

Projekt oblicza statystyki w przesuwanym oknie szeregu czasowego: średnią, wariancję i ekstremum. Każda pozycja wyniku opisuje ustalony fragment sąsiednich próbek. Podstawowa metoda bezpośrednia może wielokrotnie odczytywać te same dane, natomiast metody przyrostowe i skany ograniczają liczbę operacji. Należy porównać ich opłacalność dla różnych długości okna.

Program wczytuje jeden lub wiele szeregów i przyjmuje długość oraz krok okna. Zapisuje statystyki i pozycje okien. Należy ustalić, czy wyniki dotyczą tylko pełnych okien, oraz sposób obsługi brakujących próbek. Wersja podstawowa może wymagać kompletnych danych.

Zakres implementacji

OpenMP – Rozdzielić niezależne szeregi lub zakresy pozycji wynikowych. Dla podziału segmentowego zapewnić odczyt sąsiadujących próbek potrzebnych na brzegu.

MPI – Podzielić szereg na fragmenty z halo o długości wynikającej z okna. Wyniki składać według pozycji, bez powtórzeń okien na granicach.

CUDA lub OpenCL – Obliczać statystyki bezpośrednio w kaflach albo użyć skanów dla sum. Dla wariancji ocenić błąd metody opartej na sumach kwadratów, szczególnie przy dużej średniej.

Dodatkowe wyniki do raportu

Testować szeregi stałe, rampę, impuls i losowe przebiegi. Raportować błąd każdej statystyki, próbki/s, pamięć oraz czas w funkcji długości okna. Porównać pełen potok z kosztami halo i transferów.

Zakres zespołu

1 osoba: średnia, wariancja i jedno ekstremum dla stałego kroku. 2 osoby: dobór metody według długości okna albo przetwarzanie strumieniowe partiami.

Rozwinięcie do artykułu lub pracy magisterskiej

Czy dobór algorytmu na podstawie długości okna daje przewagę nad stałą implementacją? Porównać koszt bezpośredni i przyrostowy.